weaviate

Vetores sem Limites

Open Source para IA em Escala de Verdade

Duda Nogueira · Weaviate · Foz do Iguaçu



O caminho de hoje

1 Vetores do zero

GPS, cores e o significado de um texto

2 Weaviate na prática

busca semântica, BM25, híbrida e filtros

3 Weaviate Studio

navegue nos dados sem escrever código

4 Schema

o que o Auto Schema decide por você, e quanto custa

5 Índices vetoriais

flat, HNSW, dynamic, HFresh e MUVERA

6 Quantização

o mesmo índice com uma fração da memória

7 Cluster e observabilidade

réplicas, um nó caindo ao vivo, multi-tenancy, métricas

8 Integrações

modelos, frameworks, dados e um RAG 100% local

Tudo aqui roda num laptop, offline

Weaviate 1.40

um docker compose: cluster de 3 nós, Prometheus e Grafana

Dados reais

50 mil passagens da Wikipedia (1024 dims) e 399 páginas de papers do arXiv com multi-vetores ColQwen2

Modelo aberto

`snowflake-arctic-embed-l-v2.0` : multilíngue, roda local e reproduz os vetores do dataset (cosseno 0,96–0,98)

Todos os números saíram de notebooks executados nesta máquina. Código, notebooks e slides: **github.com/dudanogueira/latinoware-2026**

Vetores do zero

um vetor é só uma lista de números que posiciona algo num espaço

2 dimensões: Foz do Iguaçu é um vetor

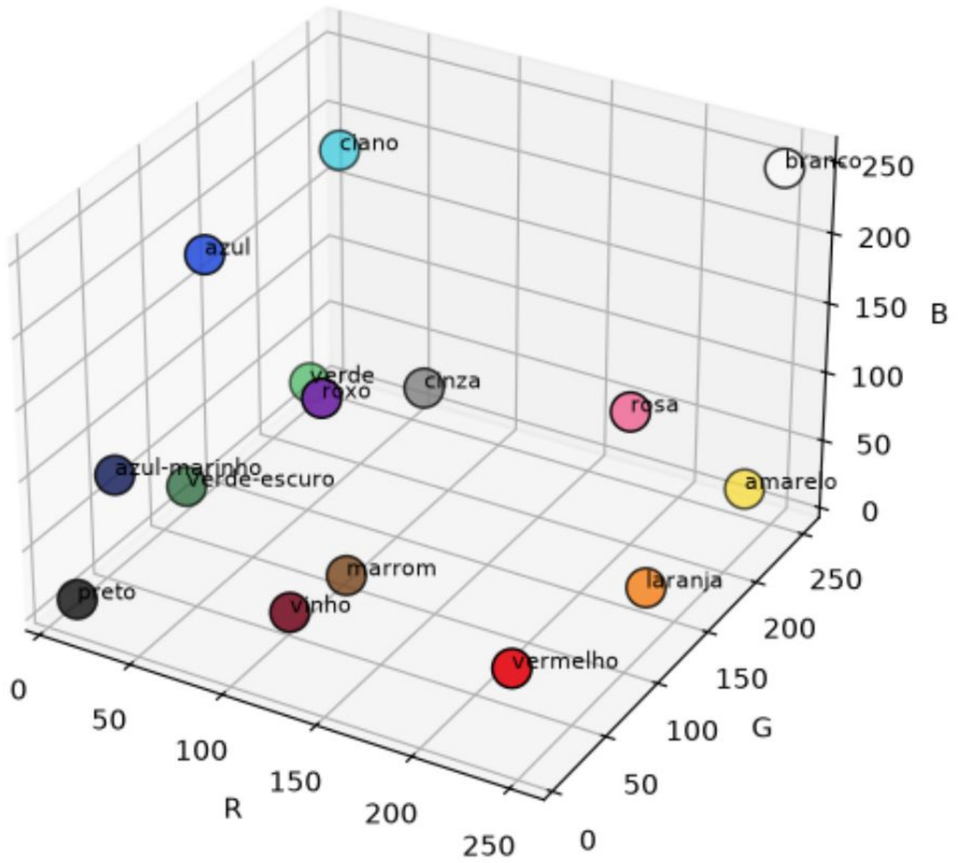
Foz = [−25,52, −54,59]

Cada cidade é um vetor [latitude, longitude]. Perto no mapa = vetores próximos.

A métrica muda os números, a ordem não. Busca por similaridade = achar os vizinhos mais próximos.

Vizinho de Foz	euclidiana	distância real
Ciudad del Este	0,03	3 km
Puerto Iguazú	0,08	9 km
Cascavel	1,26	130 km
Assunção	3,00	302 km
Curitiba	5,31	533 km

3 dimensões: cores RGB



Uma cor é um vetor [R, G, B]. Cores parecidas ficam perto no cubo.

Consulta: laranja Latinoware (240, 110, 20)

laranja	23
vermelho	83
amarelo	112

distância euclidiana no cubo RGB, 15 cores

Euclidiana × cosseno

Mesmo vermelho, brilho diferente

(100, 0, 0) × (250, 0, 0)

150
euclidiana: longe

1,0
cosseno: iguais

Cores diferentes

(250, 0, 0) × (0, 0, 250)

354
euclidiana

0,0
cosseno: ortogonais

O cosseno olha só a direção. Em embeddings de texto, o que importa é para onde o vetor aponta.

4 dimensões: CMYK

A gráfica descreve a mesma cor com 4 números. Laranja: RGB (245, 130, 30) = CMYK (0; 0,47; 0,88; 0,04).

- O número de dimensões depende de **quem codifica**, não da coisa em si
- Vetores de sistemas diferentes **não são comparáveis**: não existe distância entre um RGB e um CMYK
- Com embeddings é igual: pergunta e documentos vetorizados **com o mesmo modelo**

1024 dimensões: o significado de um texto

	1	2	3	4	5	6	7
1. cachorro latindo (pt)	1,00	0,67	0,72	0,48	0,26	0,12	0,28
2. dog barking (en)	0,67	1,00	0,73	0,43	0,26	0,18	0,27
3. perro ladra (es)	0,72	0,73	1,00	0,49	0,24	0,13	0,31
4. gato dorme (pt)	0,48	0,43	0,49	1,00	0,20	0,07	0,39
5. Cataratas do Iguaçu (pt)	0,26	0,26	0,24	0,20	1,00	0,72	0,16
6. Iguazu Falls (en)	0,12	0,18	0,13	0,07	0,72	1,00	0,08
7. trocar o óleo (pt)	0,28	0,27	0,31	0,39	0,16	0,08	1,00

cachorro · dog · perro: 0,67–0,73 sem nenhuma palavra em comum

Cataratas · Iguazu Falls: 0,72 entre português e inglês

trocar o óleo: no máximo 0,39 com o resto

> Isso é busca semântica. Não dá para desenhar 1024 dimensões, mas a matemática é a mesma do mapa.

Força bruta funciona… até não funcionar

01_vetores_do_zero.ipynb

```python
matriz = np.vstack(wiki.vector)        # (25000, 1024)
matriz /= np.linalg.norm(matriz, axis=1, keepdims=True)

q = embed_query("qual é a maior cachoeira do mundo?")
scores = matriz @ q                    # compara com TODOS
top = np.argsort(-scores)[:3]
```

10 ms
25 mil · 0,1 GB

396 ms
1 milhão · 4,1 GB

40 s
100 milhões · 410 GB

6,6 min
1 bilhão · 4 TB de RAM

medido com 25.000 vetores (9,9 ms), extrapolado linearmente

E ainda faltam persistência, filtros, update e delete, réplicas, backup, controle de acesso… É para isso que existe um banco de dados vetorial.

Weaviate na prática

banco de dados vetorial open source (BSD-3), escrito em Go

Uma collection, 25 mil objetos

02_primeiros_passos_weaviate.ipynb

```
client = weaviate.connect_to_local()

wiki = client.collections.create(
    "Wiki",
    properties=[
        Property(name="title", data_type=DataType.TEXT),
        Property(name="text", data_type=DataType.TEXT),
    ],
    vector_config=Configure.Vectors.self_provided(),  # HNSW + cosseno
)

with wiki.batch.fixed_size(batch_size=500) as batch:
    for linha in dados.itertuples():
        batch.add_object(properties={...}, vector=linha.vector)
```

4,2 s

25.000 objetos importados no cluster, 0 falhas

Vetores prontos aqui; o Weaviate também vetoriza sozinho com módulos para OpenAI, Cohere, Ollama, Hugging Face, Weaviate Embeddings...

Busca vetorial: pergunta em português, textos em inglês

```
wiki.query.near_vector(embed_query("como os vulcões entram em erupção?")
```

0,563 **Volcanism**

Intrusive volcanism is when magma is forced into the rocks that make up the Earth's crust...

0,582 **Intrusion**

An intrusion is magma (melted rock) which cools and becomes solid under the Earth's surface...

0,597 **Carbon cycle**

The main ways that carbon gets into the carbon cycle are volcanoes, and the burning of fossil fuels...

distância de cosseno: menor = mais perto

BM25 procura palavras, não significado

A mesma pergunta em português…

```
wiki.query.bm25("como os vulcões entram em erupção?")
```

8,25 — **OS/2**
"os" casou com OS...

7,18 — **Mac OS X Server 1.0**

6,25 — **Hit 'Em Up**
"em" casou com "Em"

…mas imbatível em termos exatos

```
wiki.query.bm25("Merapi")
```

4,41 — **Mount Merapi**
In 2006, scientists believed that the volcano would erupt soon...

nomes, códigos, siglas, números de série

Híbrida e filtros: o melhor dos dois

hybrid(alpha=0.6): vetor + BM25

```
wiki.query.hybrid("Merapi eruption")
```

0,637	Mount Merapi
0,601	2021 Cumbre Vieja volcanic eruption
0,272	Garibaldi Volcanic Belt

alpha=1: só vetor · alpha=0: só BM25 · comece entre 0,5 e 0,75

near_vector + Filter title like “*guitar*”

```
wiki.query.near_vector(embed_query("instrumentos musicais de corda")
```

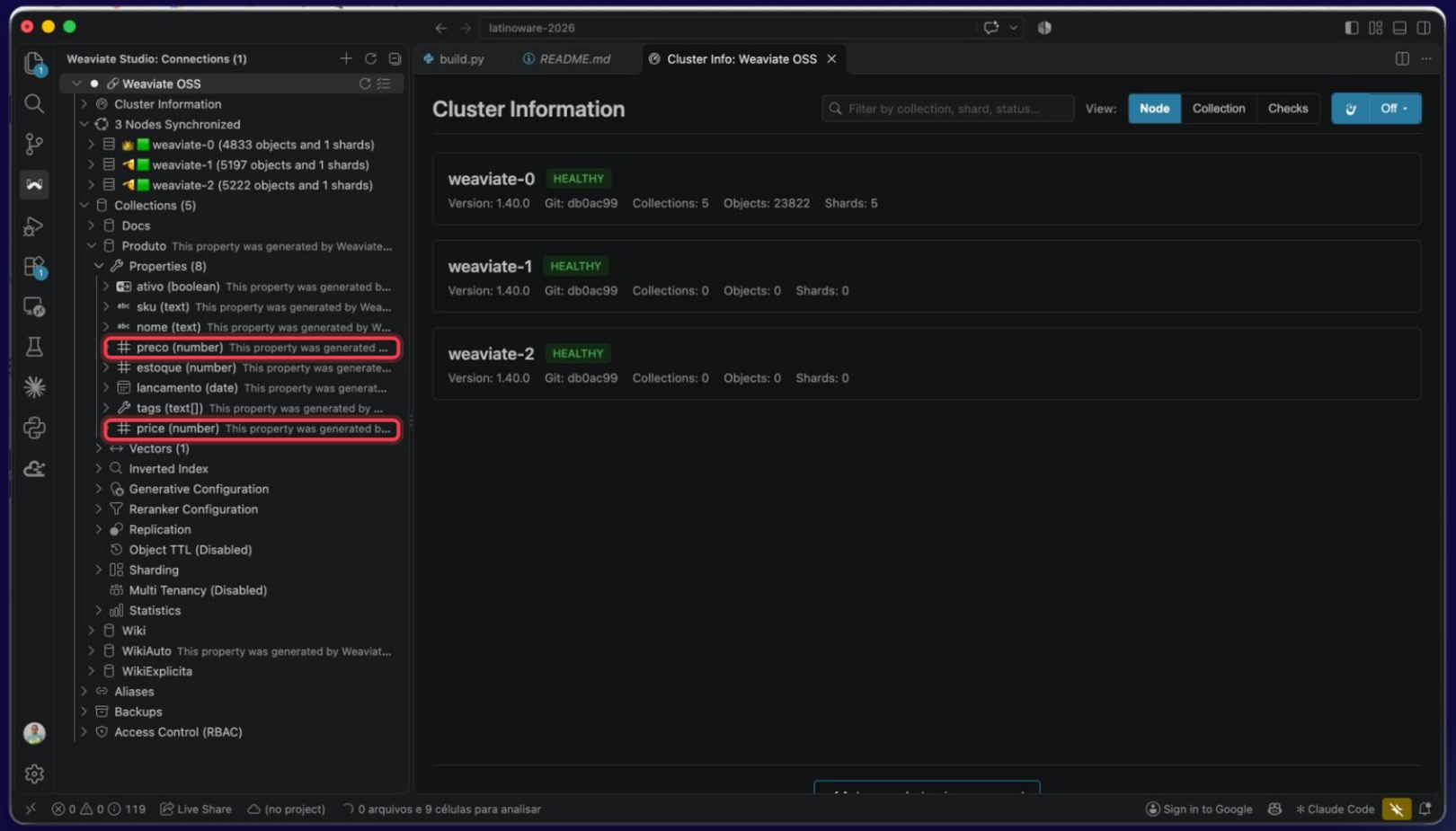
0,631	Electric guitar
0,696	Guitar
0,724	Bass guitar

o filtro é aplicado durante a travessia do índice (ACORN), sem perder resultados

Weaviate Studio

navegue nos dados sem escrever código, de dentro do editor

Weaviate Studio: o cluster dentro do editor



O cluster desta palestra: 3 nós saudáveis e, no schema de `Produto`, `preco` e `price` lado a lado.

- **Data Explorer**: objetos, filtros, busca vetorial, export
- **Schema** em árvore: propriedades, vetores, índices
- **Consultas** com resultados em tabela ou JSON
- **Operação**: nós, backups, RBAC, modo somente leitura

Instalar

`prasadmuley.weaviate-studio`

VS Code Marketplace ou Open VSX (Cursor, Windsurf)

Open source (MIT), feito pela comunidade:
github.com/muleyprasad/weaviate-studio

Studio ao vivo, nos dados desta palestra

1

Conectar

`localhost:8080`: o cluster aparece com os 3 nós e a saúde de cada um

2

Navegar

Abrir a collection `Wiki`: objetos, propriedades e o vetor de cada um

3

Buscar

Busca vetorial e filtros no Data Explorer, sem escrever código

4

Ver o problema

No schema de `Produto`: `preco` e `price` lado a lado. É o próximo assunto.

Bom para explorar e depurar; o código continua sendo a fonte da verdade do schema.

Schema: o que você não vê também custa

Auto Schema × schema explícito

Auto Schema: o primeiro objeto decide tudo

insert numa collection que não existe

```
produtos.data.insert({
    "nome": "Camiseta Latinoware 2026",
    "preco": 59,
    "estoque": 120,
    "lancamento": "2026-10-14T09:00:00Z",
    "tags": ["evento", "algodão"],
    "ativo": True,
    "sku": "LTW-2026-001",
})
```

propriedade	inferido	repare
preco, estoque	**number**	**float, não int**
lancamento	date	só porque era RFC 3339
sku	**text, word**	`ltw · 2026 · 001`
todo text	filtro + BM25	dois índices cada
tags · ativo	text[] · boolean	ok

Com tokenização word, filtrar por “2026” acha todos os SKUs que têm esse pedaço.

Três armadilhas em produção

O primeiro valor manda

`"estoque": "dez unidades"`

→ 422: invalid number property, requires a float

Acento não entra

`"preço": 39`

→ nome inválido: propriedades seguem a regra do GraphQL

Typo vira schema

`"price": 39`

→ agora existem preco e price, em silêncio. Metade dos filtros erra.

> Em produção: `AUTOSCHEMA_ENABLED=false` e schema versionado junto com o código.

Cada índice é um bucket LSM no disco

Auto Schema

232 MB

11 buckets · import 6,7 s

Explícito

171 MB

7 buckets · import 6,5 s

−26% de disco com os mesmos 25 mil artigos. Com 100 milhões, é a diferença entre caber ou não caber no nó.

Para cada propriedade, pergunte:

1. Vou filtrar por ela? → `index_filterable`
2. Vou fazer BM25/híbrida? → `index_searchable` + tokenização
3. Filtro por faixa (>, <)? → `index_range_filters`
4. Nada disso? → desligue tudo, ela só é armazenada

Índices vetoriais

recall × latência × memória × custo de escrita

Quatro índices, quatro trocas

flat

Força bruta, vetores no disco. Coleções pequenas e muitos tenants pequenos.

HNSW

Grafo em camadas, em memória. Baixa latência e alto QPS: o padrão.

dynamic

Começa flat e vira HNSW ao passar do threshold. Tenants de tamanhos variados.

HFresh (v1.40)

Posting lists no disco, só centróides em memória. Quando a RAM é o limite.

Índices vetoriais, medidos

Índice	recall@10	p50 (ms)	p99 (ms)	import + índice (s)
flat	1,000	9,87	19,12	7,3
HNSW ef=16	0,945	2,29	4,40	12,8
HNSW ef=64	0,994	3,48	7,76	12,8
HNSW ef=256	1,000	5,36	8,54	12,8
dynamic (→ HNSW)	0,998	3,68	8,70	18,6
HFresh	**1,000**	**8,75**	**23,33**	**8,0**

49.500 vetores reais de 1024 dims e 500 perguntas, com gabarito top 10 exato calculado em NumPy. Latência via gRPC, cluster local de 3 nós (Weaviate 1.40.0, ASYNC_INDEXING=true).

O que a tabela diz

ef é um botão ao vivo

ef 16 → 256: recall 0,945 → 1,000 e p50 2,3 → 5,4 ms.

Muda com `config.update()`, sem reindexar.

flat é exato, mas linear

9,9 ms com 50 mil vetores. Com milhões, cresce junto.

O HNSW cresce ~log, mas o grafo vive em memória.

HFresh: import rápido, recall alto

1,000 de recall, import em 8,0 s (flat: 7,3 s · HNSW: 12,8 s).

Troca alguns ms de latência por muito menos RAM.

HFresh por dentro

Novo na v1.40, baseado no SPFresh: os vetores são agrupados em clusters (posting lists) no disco. Na memória ficam só os centróides.

- **Centróides** em memória, comprimidos com RQ 8 bits
- **Posting lists** no disco com RQ 1 bit (até 48 KB cada)
- **Rescore** dos melhores candidatos com o vetor original (`rescore_limit=350`)
- `search_probe=256` : quantos clusters visitar por busca

Multi-vetor: um vetor por pedaço da página

399
páginas de papers de IA

300.845
vetores ColQwen2 de 128 dims

~754
vetores por página

MaxSim

Cada vetor da pergunta procura o seu melhor par no documento, e as notas são somadas. Muito preciso, muito caro.

MUVERA

Codifica o documento multi-vetor num único vetor de tamanho fixo. O índice busca nesses vetores e reranqueia com MaxSim exato.

MUVERA: 754× menos vetores no índice

Configuração	vetores no índice	import (s)	página certa em 1º	recall@10 (MaxSim)	p50 (ms)
HNSW, multi-vetor puro	300.845	31,0	100%	1,000	50,1
HNSW + MUVERA	**399**	**7,3**	**100%**	**1,000**	**54,3**
HFresh + MUVERA (v1.40)	399	6,2	100%	0,998	87,3

40 trechos de 32 vetores sorteados de uma página; a busca deve achar a página de origem. O HFresh multi-vetor reranqueia até 350 candidatos com MaxSim exato, quase a coleção inteira (399 páginas): em coleções grandes a conta muda, e o ganho dele é memória.

Quantização

o mesmo índice, uma fração da memória

O HNSW navega com os vetores em memória

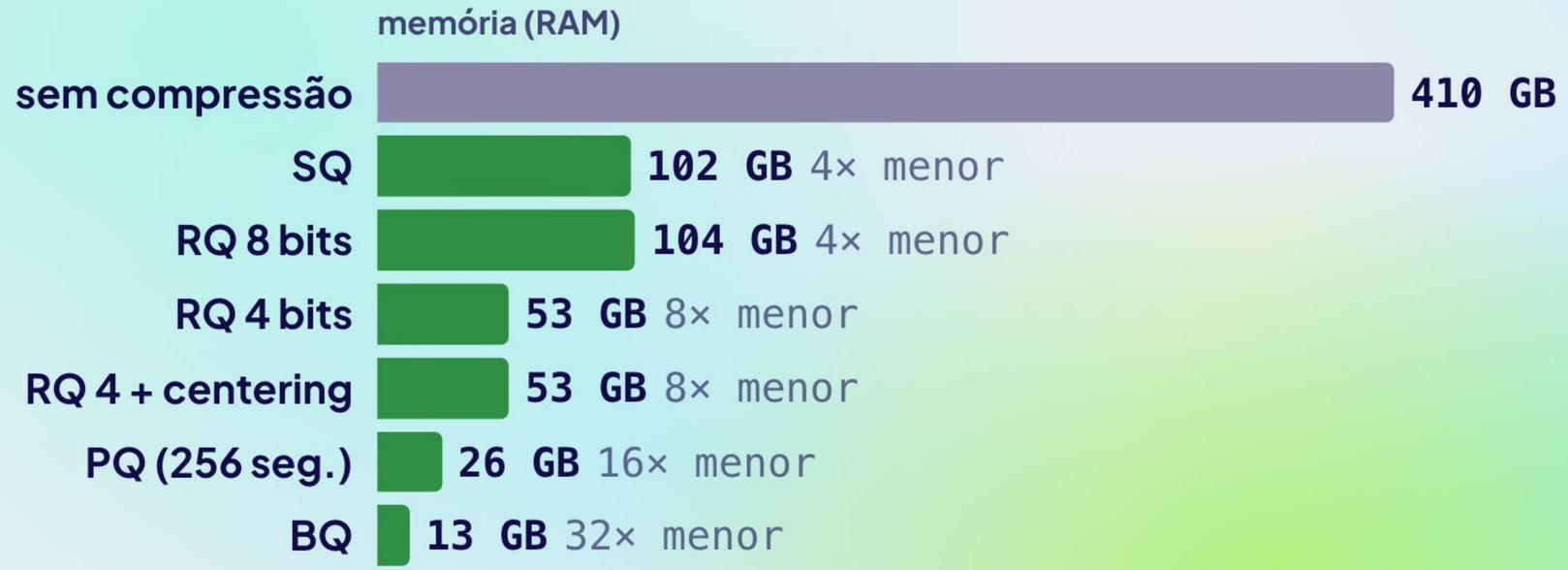
1 bilhão × 1024 dims × 4 bytes = 4 TB de RAM

Quantizar = navegar com uma versão comprimida e reavaliar (*rescore*) os melhores candidatos com o vetor original, que fica no disco.

Método	Ideia	bytes / vetor
SQ (scalar)	cada número vira 1 byte; precisa de treino	1.024
RQ 8 bits (rotational)	rotaciona e quantiza; sem treino, qualquer modelo	1.040
RQ 4 bits + centering (v1.40)	idem com 4 bits, subtraindo a média dos vetores	528
PQ (product)	segmentos + um codebook k-means por segmento	256
BQ (binary)	1 bit por dimensão: só o sinal	128

Quanta memória cada método economiza

RAM para guardar 100 milhões de vetores de 1024 dims, e quanto da busca continua certa (recall@10)



	com rescore	sem rescore
sem compressão	0,997	—
SQ	0,997	0,987
RQ 8 bits	0,998	0,994
RQ 4 bits	0,999	0,950
RQ 4 + centering	0,998	0,959
PQ (256 seg.)	0,999	—
BQ	0,994	0,994

recall@10 = fração do top 10 exato que a busca acertou, medido com 49.500 vetores reais. Rescore (o padrão) reavalia os melhores candidatos com o vetor original; “sem rescore” mostra o erro da compressão sozinha.

> RQ 4 bits: 0,950 sem rescore, 0,999 com. 8× menos memória, quase o mesmo recall.

Quantização: o que levar para casa

RQ 8 bits é a escolha segura

4× menos memória, sem treino, recall 0,998. Coleções nascem sem compressão: mude o padrão com `DEFAULT_QUANTIZATION=rq-8`.

Rescore custa disco

A qualidade volta lendo o vetor original na busca. Use SSD.

BQ: 32×, mas teste

Recall 0,994 com rescore (0,994 sem). Funciona bem com modelos grandes; meça com os seus dados.

⚠ centering precisa de async

`centering=True` só comprime com `ASYNC_INDEXING=true`. Sem isso, fica sem compressão e sem aviso.

Escala horizontal

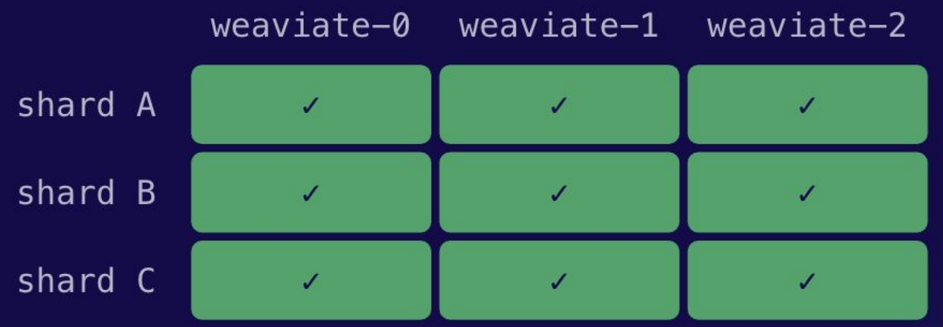
cluster de 3 nós, réplicas, multi-tenancy e métricas

Sharding + replicação

06_cluster_e_observabilidade.ipynb

```
wiki = client.collections.create(
    "WikiCluster",
    vector_config=Configure.Vectors.self_provided(
        quantizer=Configure.VectorIndex.Quantizer.rq(bits=8)),
    sharding_config=Configure.sharding(desired_count=3),
    replication_config=Configure.replication(
        factor=3, async_enabled=True),
)
```

- **Schema** via RAFT (consenso)
- **Dados**: replicação sem líder + replicação assíncrona



3 shards × 3 réplicas: cada nó guarda uma cópia de cada shard.

5,6 s

25.000 objetos com 3 réplicas, 0 falhas

Derrubando um nó ao vivo

1

Cluster saudável

w-0 w-1 w-2

busca → **Volcanism**

2

docker stop weaviate-2

w-0 w-1 ~~w-2~~

busca → **Volcanism**

3

Escrita com o nó fora

w-0 w-1 ~~w-2~~

insert “Latinoware 2026” → **aceito**

4

docker start weaviate-2

w-0 w-1 w-2

recebe o objeto **~29 s** depois

A replicação assíncrona compara árvores de hash entre réplicas e transfere só o que difere. Nenhuma intervenção manual.

Multi-tenancy: um shard por cliente

```python
notas = client.collections.create(
    "Notas",
    vector_config=Configure.Vectors.self_provided(
        vector_index_config=Configure.VectorIndex.dynamic()),
    multi_tenancy_config=Configure.multi_tenancy(
        enabled=True, auto_tenant_creation=True,
        auto_tenant_activation=True),
    replication_config=Configure.replication(factor=2),
)
notas.with_tenant("cliente-007").query.near_vector(q)
```

100
tenants criados

50 / 50
ativos / inativos

- Shards ativos por nó: 33 · 31 · 36
- Isolamento, e deleção de um cliente é trivial (LGPD!)
- Inativos saem da memória; com offload-s3, vão para o S3

Observabilidade: :2112/metrics

PROMETHEUS_MONITORING_ENABLED=true, cluster de 3 nós sob carga de busca vetorial + BM25

Sinal	PromQL	valor
nós no ar	`count(up{job="weaviate"} == 1)`	3
objetos (todas as réplicas)	`sum(object_count)`	918.700
consultas/s	`sum(rate(queries_durations_ms_count[1m]))`	455
latência p95	`histogram_quantile(0.95, … queries_durations_ms_bucket …)`	9,5 ms
fila de indexação assíncrona	`sum(queue_size)`	18
heap Go por nó	`go_memstats_heap_inuse_bytes`	1.329–1.512 MB
falhas de compactação LSM	`sum(weaviate_lsm_bucket_compaction_failure_count)`	0

Dashboards Grafana (:3000): Overview, Querying, Async Indexing, Cluster & RAFT, Replication.

O que alertar em produção

Nó fora

`up == 0`

Latência

p95 / p99 acima do SLO

Fila de indexação

crescendo sem parar

Memória

heap perto do `GOMEMLIMIT`

LSM

falhas de compactação ou flush

Disco

acima de ~80%

Integrações

o banco vetorial no meio do ecossistema open source

Onde o Weaviate se conecta

Modelos: uma linha de config

Ollama, Hugging Face, OpenAI, Cohere, Google, AWS Bedrock, Mistral, Voyage AI, Jina AI, NVIDIA, Weaviate Embeddings

Frameworks de LLM e agentes

LangChain, LlamaIndex, Haystack, DSPy, CrewAI, Semantic Kernel

Dados e orquestração

Unstructured, Airbyte, Kafka / Confluent, Spark, dlt, Kestra

Clientes e apps

Clientes Python, TypeScript, Go, Java e C#. Apps open source como AnythingLLM e mem0 já falam Weaviate.

RAG 100% local: Weaviate + Ollama

07_rag_local_ollama.ipynb

```python
OLLAMA = "http://host.docker.internal:11434"
docs = client.collections.create(
    "Docs",
    vector_config=Configure.Vectors.text2vec_ollama(
        api_endpoint=OLLAMA, model="bge-m3"),  # multilíngue
    generative_config=Configure.Generative.ollama(
        api_endpoint=OLLAMA, model="llama3.2"),
)
docs.data.insert_many([{"text": t} for t in textos])  # o Weaviate vetoriza
wait_for_indexing(client, "Docs")  # ASYNC_INDEXING: espere o índice

r = docs.generate.near_text(
    query="como os vulcões entram em erupção?", limit=3,
    grouped_task="Responda em português usando só estes trechos")
print(r.generative.text)
```

Sem chave de API

Embeddings e LLM rodando no laptop, ao lado do cluster.

Troque numa linha

`text2vec_ollama` → `text2vec_huggingface`, `text2vec_cohere` ...

Para levar para casa

1 Mesmo modelo

vetor é coordenada: pergunta e documentos no mesmo espaço

2 Híbrida + filtros

significado e termos exatos na mesma consulta

3 Schema explícito

só os índices que você usa: −26% de disco

4 Índice certo

HNSW por padrão, HFresh quando a RAM limita, MUVERA no multi-vetor

5 Quantize

RQ 8 bits: 4× menos memória, recall 0,998

6 Réplicas + métricas

um nó caiu e ninguém percebeu, a não ser o Grafana

Valeu! Perguntas?

Slides, notebooks e docker compose

github.com/dudanogueira/latinoware-2026

Duda Nogueira

- in linkedin.com/in/dudanogueira
- gh github.com/dudanogueira
- @ duda@weaviate.io

Weaviate

- weaviate.io
- gh github.com/weaviate/weaviate
- docs.weaviate.io
- forum.weaviate.io